Understanding API Security

API security focuses on protecting APIs from malicious attacks by implementing **authentication, authorization, encryption, monitoring,** and other security controls.

Introduction

Application Programming Interfaces (APIs) are essential components of modern software systems. They allow different applications to communicate and exchange data efficiently.

APIs are widely used in **mobile applications, web services, cloud platforms,** and **enterprise systems.**

However, because APIs expose system functionality and data to external users and applications, they become **attractive targets for attackers.**

Introduction

The Risk

Without proper security mechanisms, APIs can lead to:

- Data breaches
- Unauthorised access
- System compromise

The Solution

API security focuses on protecting APIs from malicious attacks by implementing:

- Authentication
- Authorisation
- Encryption
- Monitoring
- Other security controls

Importance of API Security

API security is important because APIs often handle:

What APIs Handle

- *Sensitive user data*
- *Authentication credentials*
- *Financial transactions*
- *Personal information*
- *Business-critical operations*

What Attackers Can Do

- *Access confidential data*
- *Manipulate system operations*
- *Perform unauthorised transactions*
- *Disrupt services*

Therefore, organisations must implement **strong security practices** to protect APIs.

Common API Security Threats

APIs face several common security threats including:

- Broken Authentication
- Broken Authorisation
- Injection Attacks
- Lack of Rate Limiting
- Data Exposure
- Man-in-the-Middle Attacks
- Improper Asset Management

Broken Authentication

Broken authentication occurs when APIs do not properly verify user identity. Attackers may exploit weak authentication mechanisms to gain unauthorised access.

Examples Include

- Weak passwords
- Hardcoded credentials
- Session tokens that do not expire

Example Scenario

An API allows users to log in using username and password. If the API does not limit login attempts, attackers can perform **brute force attacks** to guess passwords.

Impact

- Unauthorised access to user accounts
- Data theft
- Account takeover

Broken Authorization

Broken authorisation occurs when users can access resources they are not permitted to use.

Example Scenario

A banking API provides an endpoint:

```
GET /account/12345
```

If the API does not verify the identity of the requester, another user may access someone else's account data simply by **changing the account ID.**

Impact

- Exposure of sensitive information
- Unauthorised data modification
- Privacy violations

Injection Attacks

Injection attacks occur when attackers insert malicious input into API requests that are processed by backend systems.

Common Injection Types

- SQL Injection
- Command Injection
- NoSQL Injection

Example Scenario

An API request:

```
GET /users?id=105
```

If the backend directly executes the input without validation, an attacker might send:

```
GET /users?id=105 OR 1=1
```

Impact

- Database manipulation
- Data leaks
- Complete system compromise

Lack of Rate Limiting

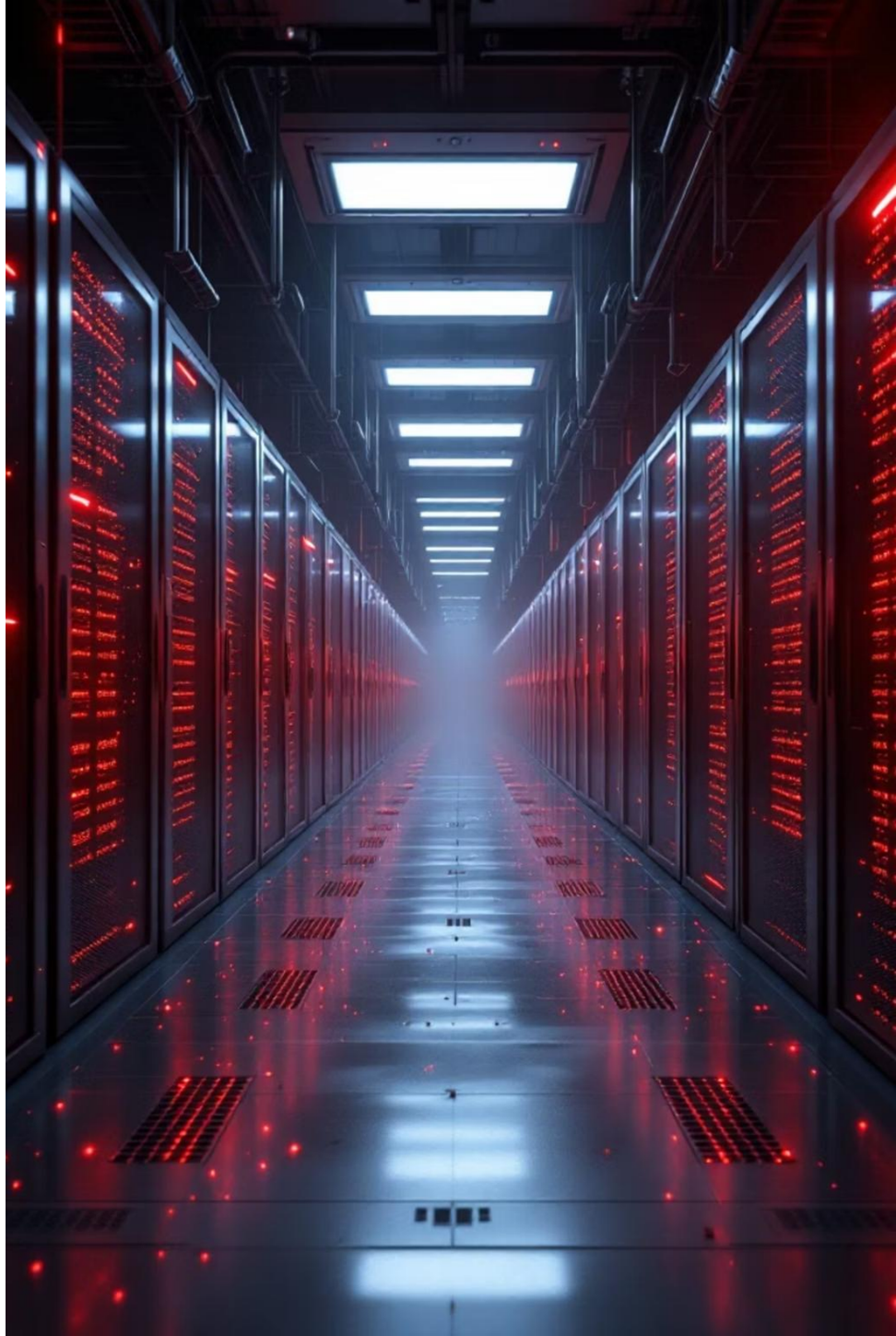
If APIs do not limit the number of requests a user can make, attackers may overwhelm the system.

Example Scenario

Attackers send **thousands of requests per second** to an API endpoint.

Impact

- Server overload
- Denial of Service (DoS)
- Reduced system availability



Data Exposure

APIs sometimes expose more information than necessary in their responses.

Example Scenario

An API response may include:

- Password hashes
- Internal system IDs
- Debug information

Impact

- Sensitive information disclosure
- Increased attack surface

Man-in-the-Middle Attacks

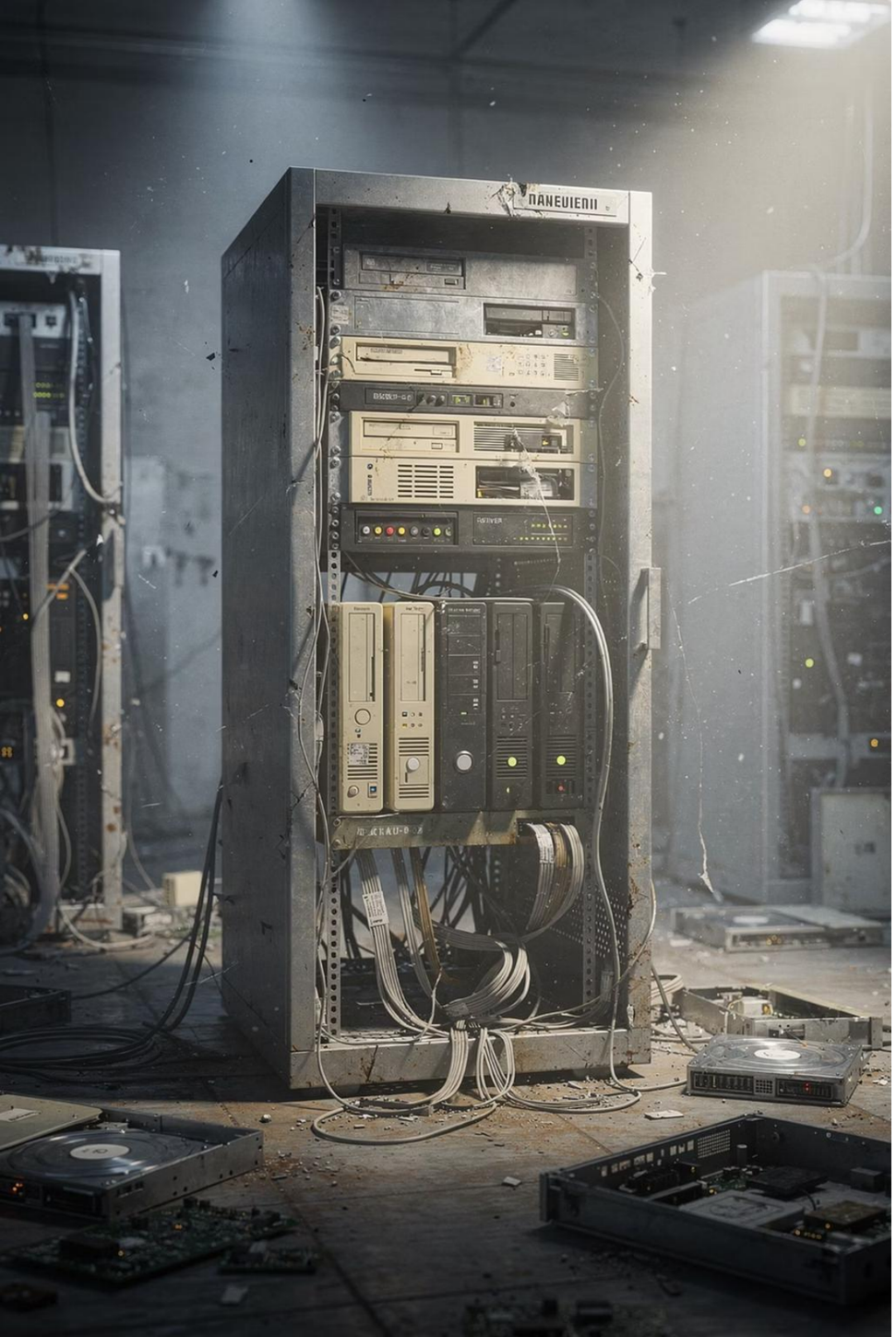
If APIs transmit data without encryption, attackers can intercept communication between clients and servers.

Example Scenario

Data transmitted using **HTTP** instead of **HTTPS**.

Impact

- Theft of login credentials
- Data manipulation
- Session hijacking



Improper Asset Management

Organisations often forget to secure older API versions.

Example Scenario

A deprecated API endpoint remains active without security controls.

Impact

- Attackers exploit outdated APIs
- Security bypass

Mitigation Strategies

To protect APIs from security threats, organisations implement various mitigation strategies.

4.1 Strong Authentication

JWT, OAuth 2.0, MFA

4.2 Proper Authorization

RBAC, ABAC

4.3 Input Validation

Filtering, sanitisation

4.4 Rate Limiting

Throttling, gateways

4.5 Encryption

TLS, AES, RSA

4.6 Logging & Monitoring

Alerts, intrusion detection

4.7 Version Management

Retire legacy APIs

Strong Authentication

Authentication ensures that users are who they claim to be.

Common authentication methods include:

JSON Web Tokens (JWT)

OAuth 2.0

Multi-factor Authentication (MFA)

Example

A user logs in and receives a JWT token:

```
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9
```

The token must be included in every API request.

Benefit: Prevents unauthorised access.

Proper Authorisation

Authorisation ensures users can only access resources they are allowed to use.

Techniques Include

- Role-Based Access Control (RBAC)
- Attribute-Based Access Control (ABAC)

Roles

- Admin
- User
- Manager

Example

An endpoint such as:

```
DELETE /users/{id}
```

should only be accessible by administrators.

Benefit: Prevents unauthorised actions.

Input Validation and Sanitization

APIs must validate all user inputs before processing them.

Techniques Include

- → Input filtering
- → Parameter validation
- → Prepared statements for database queries

Example

Accept only numeric values for user ID.

```
GET /users?id=123
```

Reject requests containing malicious characters.

Benefit: Prevents injection attacks.

Rate Limiting and Throttling

Rate limiting restricts the number of requests a client can make within a specific time period.

100

Requests per Minute

Maximum allowed per client

Tools Used

- API gateways
- Load balancers

Benefit: Protects against denial-of-service attacks.

Encryption of Data

Data should be encrypted both in transit and at rest.

Data in Transit

Using HTTPS with TLS encryption.

Example

```
https://api.example.com/users
```

Data at Rest

Sensitive data stored in databases should be encrypted using algorithms such as:

- AES
- RSA

Benefit: Protects confidential data.

Logging and Monitoring

Continuous monitoring helps detect suspicious behaviour.

Techniques Include

- API logging
- Security alerts
- Intrusion detection systems

Example

Monitor repeated failed login attempts.

Benefit: Early detection of attacks.

API Version Management

Maintain proper documentation and deactivate outdated APIs.

Example

```
/api/v1/users
/api/v2/users
```

Older versions should be securely retired.

Benefit: Prevents exploitation of legacy APIs.

Example Use Case: Online Banking API

Consider an online banking system that provides APIs for mobile applications.

Threat

Attackers attempt to access another user's account by changing account IDs in API requests.

Example Attack

```
GET /accounts/56789
```

Banking API Mitigation Strategy

MITIGATION STRATEGY

Authentication

Users must log in and obtain a JWT token.

```
POST /login
```

Authorization

API verifies that the authenticated user owns the requested account.

Encryption

All communication uses HTTPS.

Rate Limiting

Limit requests to prevent brute force attacks.

Logging

Suspicious activities are logged and monitored.

Result: The banking API becomes more secure against unauthorised access and data breaches.

Best Practices for API Security

Organisations should follow these best practices:

1. *Use strong authentication mechanisms*
2. *Implement role-based access control*
3. *Validate all user inputs*
4. *Use HTTPS for all API communications*
5. *Encrypt sensitive data*
6. *Implement rate limiting*
7. *Monitor API usage*
8. *Regularly update and patch systems*

Conclusion

APIs play a crucial role in modern software architectures, but they also introduce **significant security risks** if not properly protected.

Understanding the threats that APIs face and implementing appropriate mitigation strategies is **essential for maintaining system integrity** and protecting sensitive data.

By applying **strong authentication, authorisation, encryption, input validation, and monitoring practices**, organisations can significantly reduce API vulnerabilities and ensure secure communication between applications.